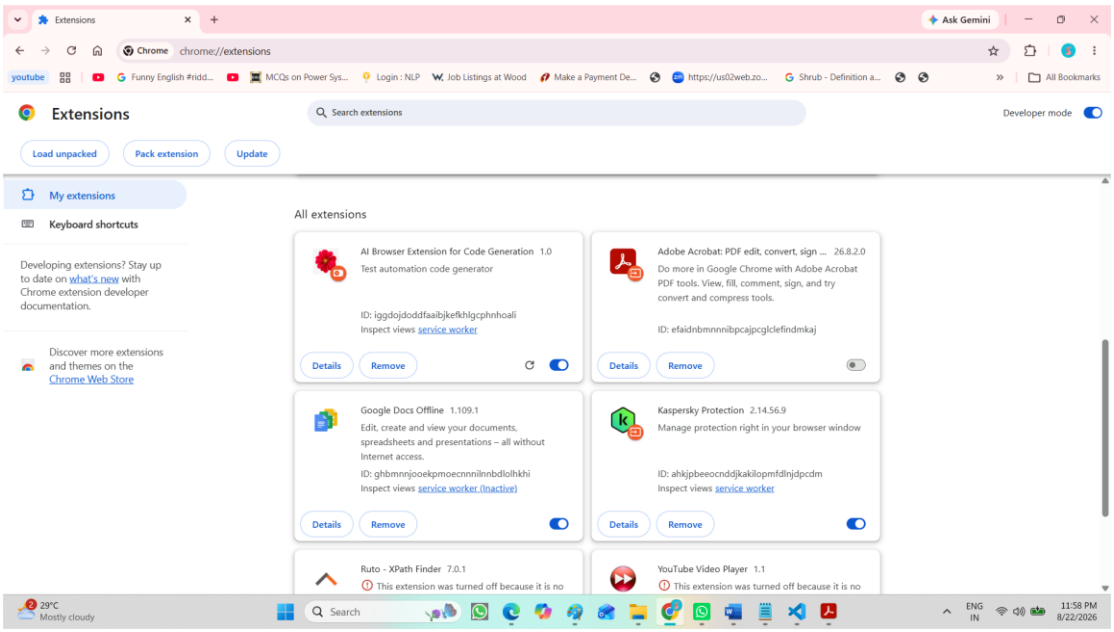
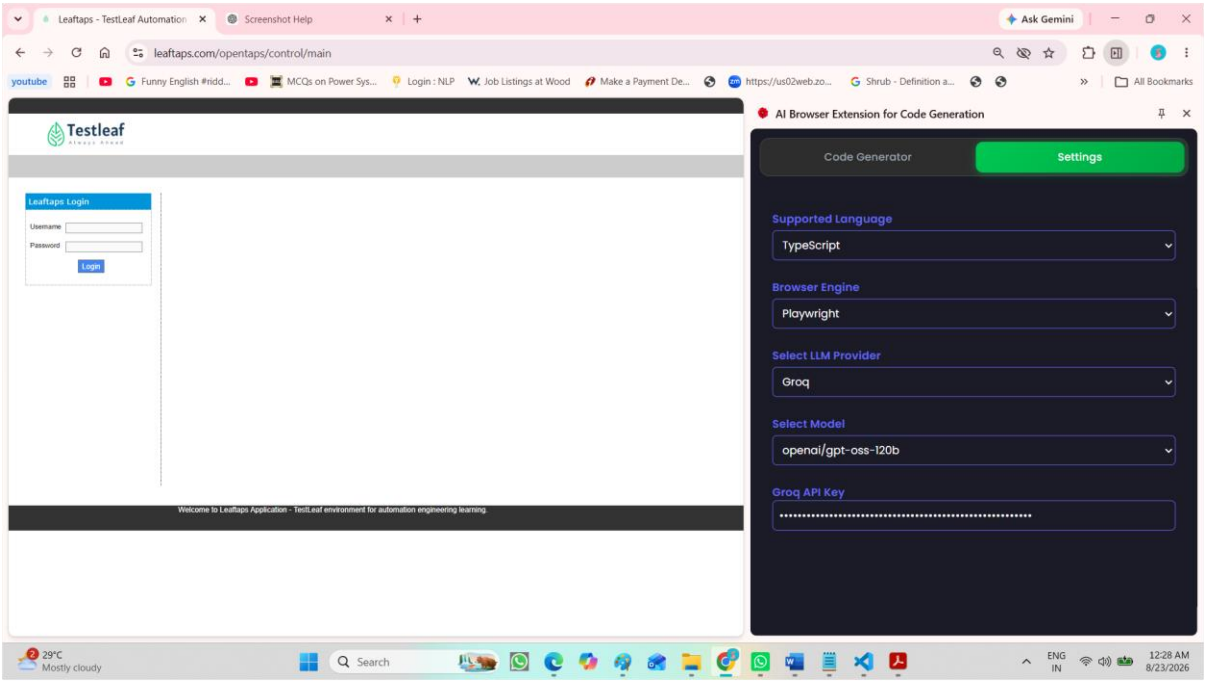
1) **To change log**

- Image is updated with correct resolution
- Updated in appconfig.js and manifest.json

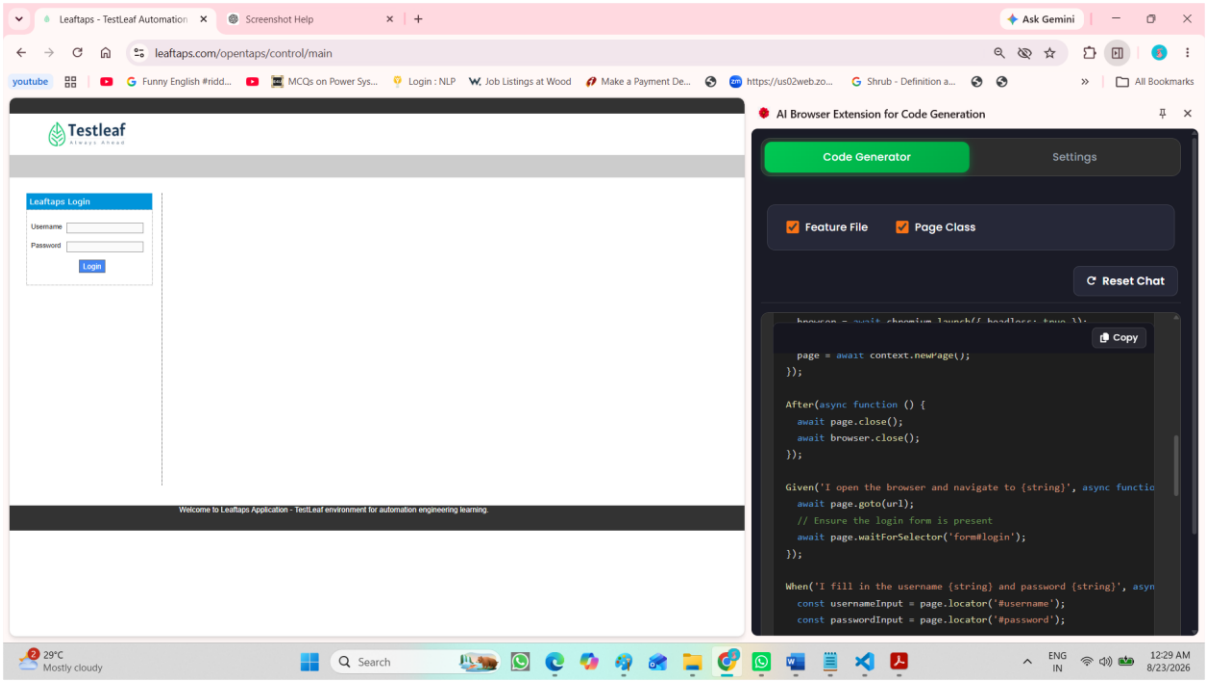
Output-



2) Add "openai/gpt-oss-120b" to the existing list



3) **Implement Playwright with TypeScript**




```
import { Given, When, Then, Before, After } from '@cucumber/cucumber';
import { chromium, Browser, Page } from 'playwright';
import assert from 'assert';

let browser: Browser;
let page: Page;

Before(async function () {
  browser = await chromium.launch({ headless: true });
  const context = await browser.newContext();
  page = await context.newPage();
});

After(async function () {
  await page.close();
  await browser.close();
});
```

```
Given('I open the browser and navigate to {string}', async function (url: string) {
  await page.goto(url);
  // Ensure the login form is present
  await page.waitForSelector('form#login');
});

When('I fill in the username {string} and password {string}', async function (username: string, password: string) {
  const usernameInput = page.locator('#username');
  const passwordInput = page.locator('#password');

  // Clear any existing text
  await usernameInput.fill('');
  await passwordInput.fill('');

  // Fill fields only if values are provided (to test required validation)
  if (username !== '') {
    await usernameInput.type(username);
  }
  if (password !== '') {
    await passwordInput.type(password);
  }
});

When('I submit the login form', async function () {
  const submitButton = page.locator('input.decorativeSubmit');
  await Promise.all([
    page.waitForNavigation({ waitUntil: 'domcontentloaded' }).catch(() => {}), // navigation may not happen on error
    submitButton.click(),
  ]);
});
```

```
Then('I should see the result {string}', async function (expected: string) {
  // Define possible selectors for success and error messages
  const successMessage = page.locator('text=Welcome to the Dashboard');
  const errorMessage = page.locator('text=Invalid login credentials');
  const usernameError = page.locator('text=Username is required');
  const passwordError = page.locator('text=Password is required');

  // Wait for any of the messages to appear
  await Promise.race([
    successMessage.waitFor({ state: 'visible', timeout: 3000 }).catch(() => {}),
    errorMessage.waitFor({ state: 'visible', timeout: 3000 }).catch(() => {}),
    usernameError.waitFor({ state: 'visible', timeout: 3000 }).catch(() => {}),
    passwordError.waitFor({ state: 'visible', timeout: 3000 }).catch(() => {}),
  ]);

  // Gather visible text
  const visibleTexts = await page.evaluate(() => {
    return Array.from(document.body.querySelectorAll('*'))
      .filter(el => el instanceof HTMLElement && el.offsetParent !== null)
      .map(el => el.textContent?.trim())
      .filter(Boolean);
  });

  const match = visibleTexts.some(text => text?.includes(expected));
  assert.ok(match, `Expected to find "${expected}" on the page, but it was not present.`);
});
```